Project Remote Control

Name: Lam Bing Qing Andrew
Student: s5
Unit: CFC060524
Trainer: Samson

Table of Contents:

1. Introduction

2. Script
   - 2.1. Functions
     - pcheck
     - uinput
     - servicechk
     - sshp
     - ftpp
     - scan
   - 2.2. Getting remote server details from user
   - 2.3. Checking and installing required packages from apt-get
   - 2.4. Checking and installing NIPE
   - 2.5. Running NIPE
   - 2.6. User input for domain/IP address to scan
   - 2.7. Checking accessibility of remote server
   - 2.8. Connecting to remote server
   - 2.9. Scans and logging

3. Discussion: FTP

4. Conclusion

5. Recommendations

6. References

1. Introduction

This report aims to explain the accompanying script and what commands and functions were used to achieve its objectives. It also aims to showcase how the script can be used to automate an attack on remote servers.

The script is configured to execute commands on a remote server and retrieve the results without much input from the user. The user only has to ensure they have provided the correct remote server details and domain/IP address to scan; the script would do the rest.

2. Script

2.1. Functions

Functions allows users to store and organize commands. In this script, they are used to repeat commands and make the code look neater. The explanations of the individual functions used in the script are listed below.

- `pcheck`

```
23  #~ Function pcheck is called to check for whether the required packages are installed, and if they are not,
24  #~ install them from apt-get. Only limited to packages available from apt-get.
25  function pcheck()
26  {
27
28      chkfor=$(whereis $1 | awk '{print $2}')
29
30      if [ -z "$chkfor" ]
31      then
32
33          echo "[*] $2 is not installed, installing now..."
34          echo
35          sudo apt-get -qq install $2
36          echo
37          echo "[*] $2 installed."
38
39      else
40
41          echo "[#] $2 is already installed."
42
43      fi
44
45  }
46
```

`pcheck` is used to check if the provided package is installed. If it isn't, `pcheck` installs the package.

```
[*] geoip-bin is not installed, installing now...

Selecting previously unselected package geoip-bin.
(Reading database ... 391259 files and directories currently installed.)
Preparing to unpack .../geoip-bin_1.6.12-11.1_amd64.deb ...
Unpacking geoip-bin (1.6.12-11.1) ...
Setting up geoip-bin (1.6.12-11.1) ...
Processing triggers for kali-menu (2023.4.7) ...
Processing triggers for man-db (2.12.1-1) ...

[*] geoip-bin installed.
```

```
[#] geoip-bin is already installed.
```

It does this by first checking for the provided package using:

```
chkfor=$(whereis $1 | awk '{print $2}')
```

The command uses `whereis` and passes the first argument of the function(`$1`) to it. `whereis` is a command which lists out the binary, source, and manual files of a command. Doing so, `whereis` would provide the following output:

```
┌──(kali㉿kali)-[~/Desktop]
└─$ whereis sshpass
sshpass: /usr/bin/sshpass /usr/share/man/man1/sshpass.1.gz
```

Afterwards, the output is passed through `awk`, which can be used to filter the output according to patterns or keywords, to print only the directory of the command, which is the second field, using the argument `'{print $2}'`. On the command line, it should look like this:

```
┌──(kali㉿kali)-[~/Desktop]
└─$ whereis sshpass | awk '{print $2}'
/usr/bin/sshpass
```

This output is then saved into the variable `$chkfor` to be used in the next step.

Incidentally, here is what the output would look like if the command is not installed on the system:

```
┌──(kali㉿kali)-[~/Desktop]
└─$ whereis sshpass | awk '{print $2}'
```

As can be seen, the output is empty. This would be useful in the next step.

The variable is then passed through an `if`/`else` statement to check if it is empty by using the `[ -z <variable> ]` condition. `[ ]` denotes a condition and the `-z` flag checks for an empty string. Using this, in combination with the output from the previous commands, The script can determine if the package is installed on the machine.

If the package is not installed on the machine, the script then informs the user of that fact and that it is going to install the package. It does so by using the `echo` command, which outputs the prepared line onto the command line. `"<string>"` tells `echo` what to output.

The next command is:

```
sudo apt-get -qq install $2
```

This is the command that installs the package. `sudo` has to be used here as installing packages requires elevated permissions. `apt-get` is the command to access the package manager. The script tells `apt-get` to install the package by using the `install` argument, and uses the `-qq` flag to tell it to minimize unnecessary output. `-qq` also informs `apt-get` to answer yes for any verification prompts automatically. The script passes on its second argument(`$2`) to `apt-get` to tell it what package to install. Note that `$2` here is different from the `$2` used in `awk`. `awk` understands `$2` as the second field

of the output passed on to it, while here, `$2` is treated as the second argument of the function `pcheck`.

Also of note here is that despite using the `-qq` flag, `apt-get` would still generate some output when installing packages. This is partly due to `dpkg`, which `apt-get` calls on during the installation process, preparing the packages downloaded by `apt-get`. It is possible to suppress the output from `dpkg` as well, however it is not done in this case as not only is it a low priority matter, it is also to let the user stay informed on what is being done on their machine. `-qq` is used as otherwise the output would be very long for each package that needs to be installed.

After the package is installed, `echo` is used again to inform the user.

Otherwise, if the package is installed, the `if`/`else` statement then runs the commands under the `else` section, which is a simple informing of the user using `echo`. The function then ends.

- `uinput`

```
47  #~ Function uinput is called to get a user input and verify whether said input is valid.
48  #~ If not, uinput would continue asking for a valid input until one is given or the user force quits out of the script.
49  function uinput()
50  {
51
52      read -p "[?] Specify a domain/IP address to scan: " input
53      echo
54
55      ipvalid="^(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5]))\.){3}(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5])))$"
56      domvalid="^([a-zA-Z0-9]|[a-zA-Z0-9][a-zA-Z0-9-]{0,61}[a-zA-Z0-9])\.+[a-zA-Z]{2,}$"
57
58      if [[ $input =~ $ipvalid || $input =~ $domvalid ]]
59      then
60
61          echo "[*] Input accepted. Connecting to remote server:"
62
63      else
64
65          echo "[!] Not a valid domain or IP address! Please input a valid one."
66          uinput
67
68      fi
69
70  }
71
```

The function `uinput` is used to get and check the user input when prompting the user for a domain or IP address. The reason this is a function is we can loop it to get the user input again if the provided input is invalid.

```
[?] Specify a domain/IP address to scan: 152.42.232.203

[*] Input accepted. Connecting to remote server:
```

```
[?] Specify a domain/IP address to scan: !!!.{}:.x.com

[!] Not a valid domain or IP address! Please input a valid one.
```

The function starts by asking for a user input using the `read` command. Using the `-p` flag, we can also output a prompt at the same time. The input is then saved into the `input` variable.

We then define two variables, `$ipvalid` and `$domvalid` as such:

```
ipvalid="^(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5]))\.){3}(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5])))$"
domvalid="^([a-zA-Z0-9]|[a-zA-Z0-9][a-zA-Z0-9-]{0,61}[a-zA-Z0-9])\.+[a-zA-Z]{2,}$"
```

We are using regular expressions(regex) here to define the structure of an IPv4 address and domain name, and saving them to `$ipvalid` and `$domvalid`, respectively.

For `$ipvalid`, we are defining the structure of an IPv4 address to be `X.X.X.X`, with each X being a digit, and each X's value falling in the range of 0 ~ 255.

For `$domvalid`, we are defining the domain name structure to be `<optional subdomains>.<domain>.<top level domain>`. Each domain/subdomain sections are from 1 to 63 characters in length, with only letters or digits allowed for at the start and end of the domain/subdomain, and letters, digits, and dashes allowed in between. Each domain/subdomain section then has to be followed by a period(`.`). The user can key in as many subdomains as they like, but the address has to contain at least one domain. It then ends with a top level domain(TLD). We are not checking for a valid TLD as the list of valid TLDs is really long and we are not trying to make this script too long. Instead, we check if the TLD consists of only alphabets and is at least 2 characters in length. The list of valid TLDs can be found here:
List of Internet top-level domains - Wikipedia

After defining the structures of both IPv4 address and domain names, we then use an `if`/`else` statement to check the user input against the structures.
A `[[ <variable> =~ <regex> ]]` conditional has to be used here to check for regex matches, with `[[ ]]` being the condition command, and `=~` being the operator that checks the user input against the variables containing our regex structures. Note that for regex matches, we have to use `[[ ]]` instead of `[ ]` as `[ ]` does not support regex matching.

If the user input is valid, the script informs the user and proceeds to the next step. However, if the input in invalid, the script informs the user of the invalid input and loops the function by calling on itself. It is looped until a valid user input is detected.

- `servicechk`

```
72  #~ Function servicechk checks that the remote server services are running. If not, it exits the script.
73  function servicechk()
74  {
75
76      if [ -z $2 ]
77      then
78
79          echo
80          echo "[!] Remote server $1 service is not running! Exiting..."
81          exit
82
83      fi
84
85  }
86
```

`servicechk` is a simple function that takes two arguments, the service name and port, and checks the second one. If it is empty, it informs the user that the remote server service is not running and exits the script using the `exit` command.

```
[!] Remote server SSH service is not running! Exiting...
```

- `sshp`

```
87  #~ Function sshp is used to run commands on the remote server.
88  #~ Only reason it exists is because I am lazy to type the same long command over and over.
89  function sshp()
90  {
91
92      sshpass -p "$pass" ssh -o StrictHostKeyChecking=no -p "$sshport" "$user"@"$host" "$@"
93
94  }
95
```

`sshp` is a function with only one command. It takes the function argument and using `sshpass`, along with the provided remote server details and ports, passes the arguments through to the remote server, using `ssh`, to execute.

The command starts by using `sshpass`, and using the `-p` option, we can tell `sshpass` what password to use with the remote server. The next option, `ssh`, is the service we would like to have `sshpass` use. It is also actually the `ssh` command, so from here we are using the syntax for the `ssh` command.

`-o StrictHostKeyChecking=no` tells the machine that we do not want to be prompted for a first connection to a remote server, and automatically adds the remote server to the list of known hosts. Using `-p` again, but this time for the `ssh` command, we can tell `ssh` which port to connect to. Next is the username and IPv4 address of the remote server to tell `ssh` what with and where to connect.

Lastly, we close out by telling sshpass what commands we want to run on the remote server by enclosing them in the `" "`.

- `ftpp`

```
96  #~ Function ftpp is used to grab the required files from the remote server.
97  #~ Only reason it exists is because I do not want the code to look too messy.
98  function ftpp()
99  {
100
101     ftp -n -P "$ftpport" "$host" &> /dev/null <<EOF
102     user "$user" "$pass"
103     get "$@"
104     bye
105 EOF
106
107 }
108
```

`fftp` is used to connect to the remote server using the FTP service and downloading the scan results back onto the local machine. It starts by calling the `ftp` command, then using the `-n` option, we turn off the login prompt from the remote server's ftp service. We then define the port to connect to by using the `-P` option. What follows is the remote server's IP address. As we do not want the output from the ftp server to show up, we send it off to oblivion by using `&> /dev/null`. Using `<<EOF <commands> EOF`, we then tell the machine to pass the enclosed commands on to the FTP service. We pass on the login details using `user`, and grab the file we want using `get`. We then exit out of the FTP service by using `bye`.

- `scan`

```
109 #~ Function scan is used to run the scans and save the scan results on both the remote server and local machine
110 #~ using both functions sshp and ftpp.
111 function scan()
112 {
113
114     cd ~
115     echo "[*] $1ing target's address: "
116     sshp "$@ $input > $1_$input"
117     ftpp "$1_$input"
118     sudo echo "$(date): [*] $1 data collected for: $input" >> /var/log/rc.log
119     sudo updatedb
120     echo "[@] $1 data was saved into: $(locate $1_$input)"
121     echo
122
123 }
124
```

```
[*] whoising target's address:
[@] whois data was saved into: /home/kali/whois_152.42.232.203

[*] nmaping target's address:
[@] nmap data was saved into: /home/kali/nmap_152.42.232.203

Scan attempts were logged onto /var/log/rc.log
```

This is the function that brings together `sshp` and `ftpp` to do the scans on the domain/IP address that the user has entered. The reason `sshp` is a separate function is due to it having to be called outside this function, and for `ftpp`, it was separated out to

its own function to keep the code tidy. This function starts by changing to the user's home folder with `cd ~`, then it informs the user that it is scanning the provided address using either `whois` or `nmap`, which is provided as an argument to this function.

It then passes on the arguments and provided address to `sshp`, and writes the results into a file titled `<scan method>_<provided address>` on the remote server.

After that, using `ftpp`, we download the file from the remote server. A log record kalof the scan is then added to `/var/log/rc.log`. As we have just downloaded the file off the remote server, we use `sudo updatedb` to update the file index. We then inform the user where the file was saved to.

2.2. Getting remote server details from user

```
#~ Actual script starts here. It starts with checking whether the remote server details has been set up.
#~ If not, the script tells you how to set up the details and exits.
#~ The script would also only accept a valid IPv4 address as the remote server address.
#~ And yes, the script would write the remote server details into a file on the local machine.
#~ Only way I can think of to get the script to remember the details between multiple executions.
#~ File is unencrypted, of course. Trying to make the file secure would result in the script taking too long to run every time.
#~ Not to mention the script's code is open for all to see. Only advice would be to secure your local machine.
if [ "$1" == "options" ]
then

    ipvalid="^(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5]))\.){3}(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5])))$"
    read -p "[*] Please provide the remote server IP address: " rshost

    while [[ ! $rshost =~ $ipvalid ]]
    do

        echo "[!] Invalid IP address format!"
        read -p "[*] Please provide the remote server IP address: " rshost

    done

    echo $rshost > rs_details
    read -p "[*] Please provide the remote server username: " rsuser
    echo $rsuser >> rs_details
    read -p "[*] Please provide the remote server password: " rspass
    echo $rspass >> rs_details
    echo
    echo "[#] Remote server details set up! Please bash the script again without any arguments."
    exit

else

    if [ $(cat rs_details 2> /dev/null | wc -l) -ne 3 ]
    then

        echo "[!] Setup not complete!"
        echo "[!] Please provide remote server details by running: bash rc.sh options or ./rc.sh options"

    exit
    fi

fi

#~ Grabbing remote server details from file.
host=$(cat rs_details | awk 'NR==1')
user=$(cat rs_details | awk 'NR==2')
pass=$(cat rs_details | awk 'NR==3')

#~ Do not try to cheat and change the remote server IP address details afterwards...
#~ The script can see it...
ipvalid="^(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5]))\.){3}(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5])))$"

if [[ ! $host =~ $ipvalid ]]
then

    echo "[!] Remote server IP address is invalid! Exiting..."
    exit

fi
```

```
┌──(kali㉿kali)-[~/Desktop]
└─$ bash rc.sh
[!] Setup not complete!
[!] Please provide remote server details by running: bash rc.sh options or ./rc.sh options

┌──(kali㉿kali)-[~/Desktop]
└─$ bash rc.sh options
[*] Please provide the remote server IP address: ^[[F^[[5~^[[B
[!] Invalid IP address format!
[*] Please provide the remote server IP address: 192.168.19.129
[*] Please provide the remote server username: tc
[*] Please provide the remote server password: tdc

[#] Remote server details set up! Please bash the script again without any arguments.
```

Despite all the functions defined at the start of the script, the actual script starts executing from this point. It starts by checking if there is a file called `rs_details` with

the remote server details, like username, password, and IPv4 address. If the file does not exist, or that it is missing one of the details, the script would inform the user of it and to run the script with the `options` option and exit.

If the script is run with the `options` option, the script would prompt the user for the remote server's IP address, username, and password and save them to the `rs_details` file. It would also check the validity of the IPv4 address. It then informs the user to run the script again and exits.

After setting up the remote server details, the user can then run the script. There is however one last check to make sure the IP address in the file is still valid before running the script. If it is not, the script exits.

2.3. Checking and installing required packages from apt-get

```
185  #~ Making updates to the local apt-get index files so apt-get can actually find the required packages.
186  sudo apt-get -qq update
187
188  #~ Here is where the script checks and/or installs the required packages.
189  pcheck geoiplookup geoip-bin
190  pcheck sshpass sshpass
191  pcheck cpanm cpanminus
192
```

```
[*] sshpass is not installed, installing now...

Selecting previously unselected package sshpass.
(Reading database ... 391267 files and directories currently installed.)
Preparing to unpack .../sshpass_1.10-0.1_amd64.deb ...
Unpacking sshpass (1.10-0.1) ...
Setting up sshpass (1.10-0.1) ...
Processing triggers for man-db (2.12.1-1) ...
Processing triggers for kali-menu (2023.4.7) ...

[*] sshpass installed.
```

The script then calls on `apt-get` using elevated permissions and tells it to update the local machine's package database by using the `update` option. It also tells `apt-get` to not output any messages by using the `-qq` option. We have to do this as by default, the local package database may not contain the information on the packages we are trying to install.

After updating, we call on the `pcheck` function to check whether `geoiplookup`, `sshpass`, and `cpanm` are installed. If they are, `pcheck` informs the user that they are. If they are not, `pcheck` installs the required packages automatically.

In this case, `pcheck` and `apt-get update` only works with packages that are on `apt-get`. For packages that are not, like NIPE, we need to sideload it.

2.4. Checking and installing NIPE

```
193  #~ Updating the file index so locate results are accurate.
194  sudo updatedb
195
196  #~ Checking if NIPE exists on the local machine. If it does not, install it.
197  nipechk=$(locate -w nipe.pl)
198
199  if [ -z "$nipechk" ]
200  then
201
202      echo "[*] nipe is not installed, installing now..."
203      echo
204
205      cd ~
206      git clone https://github.com/htrgouvea/nipe && cd nipe
207      sudo cpanm --installdeps .
208      yes | sudo cpan install Switch JSON LWP::UserAgent Config::Simple
209      sudo perl nipe.pl install
210
211      nipeloc=$(pwd)
212
213      echo
214      echo "[*] nipe is installed."
215      sleep 10
216
217  else
218
219      echo "[#] nipe is already installed."
220      nipeloc=$(echo $nipechk | awk -F/ '{$NF=""}1' | tr " " "/")
221
222  fi
223
```

```
[*] nipe is not installed, installing now...

Cloning into 'nipe'...
remote: Enumerating objects: 1924, done.
remote: Counting objects: 100% (395/395), done.
remote: Compressing objects: 100% (209/209), done.
remote: Total 1924 (delta 190), reused 339 (delta 159), pack-reused 1529
Receiving objects: 100% (1924/1924), 303.06 KiB | 17.83 MiB/s, done.
Resolving deltas: 100% (1003/1003), done.
```

For checking for and/or installing NIPE, we first update the file index by running `sudo updatedb`. We then use the `locate` command with the `-w` option, for matching for the nipe.pl file. We then pass the results through an `if`/`else` statement, which informs the user that NIPE is installed if it is and installs NIPE automatically if it is not.

NIPE is installed in the user's home folder, and the location of the `nipe.pl` file is saved to a variable.

2.5. Running NIPE

```
#~ Preparations to run NIPE
echo
cd $nipeloc

#~ NIPE likes to fail, especially after it has just been installed.
#~ As such, starting NIPE would be retried a maximum of 5 times to try and prevent needless exits.
for retry in {0..5}
do

    sudo perl nipe.pl restart
    nipestart=$(sudo perl nipe.pl status | head -n 2 | tail -n 1 | awk '{print $3}')

    if [ "$nipestart" == "true" ]
    then

        break

    fi

    sleep 1

done

#~ If NIPE still refuses to connect after the retries, exit the script as user is not anonymous.
if [ "$nipestart" != "true" ]
then

    echo "[!] nipe failed to start! You are not anonymous. Exiting..."
    exit

else

    echo "[*] You are anonymous. Proceeding with next steps..."

fi

#~ Displaying the spoofed IP address and country from NIPE.
spoofip=$(sudo perl nipe.pl status | tail -n 2 | head -n 1 | awk '{print $3}')
spoofloc=$(geoiplookup $spoofip | awk -F: '{print $2}' | awk '{$1=""}1')
echo "[#] Your spoofed IP address is: $spoofip, country: $spoofloc "

```

Here, the script changes to NIPE's directory and tries to start NIPE by using the `sudo perl nipe.pl restart` command. It then checks whether NIPE has succeeded in starting and saves it to a variable. The variable is then checked and if NIPE has started successfully, the script moves on to the next step.

```
[*] You are anonymous. Proceeding with next steps...
[#] Your spoofed IP address is: 185.220.101.12, country:  Germany
```

However, if NIPE failed to start, we would try to restart NIPE to a maximum of five times, using a `for` loop. If NIPE still fails to connect after five retries, the script exits.

If NIPE starts successfully, we would inform the user what their spoofed IP address and location is using NIPE.

2.6. User input for domain/IP address to scan

```
265  #~ Asking for user input for domain/IP address to scan. Limited to IPv4 addresses and valid domain names.
266  uinput
```

```
47  #~ Function uinput is called to get a user input and verify whether said input is valid.
48  #~ If not, uinput would continue asking for a valid input until one is given or the user force quits out of the script.
49  function uinput()
50  {
51
52      read -p "[?] Specify a domain/IP address to scan: " input
53      echo
54
55      ipvalid="^(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5]))\.){3}(0*(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5])))$"
56      domvalid="^(([a-zA-Z0-9]|[a-zA-Z0-9][a-zA-Z0-9-]{0,61}[a-zA-Z0-9])\.)+[a-zA-Z]{2,}$"
57
58      if [[ $input =~ $ipvalid || $input =~ $domvalid ]]
59      then
60
61          echo "[*] Input accepted. Connecting to remote server:"
62
63      else
64
65          echo "[!] Not a valid domain or IP address! Please input a valid one."
66          uinput
67
68      fi
69
70  }
71
```

For user input, we call on the `uinput` function. The function loops as long as a valid input is not detected.

```
[?] Specify a domain/IP address to scan: 152.42.232.203

[*] Input accepted. Connecting to remote server:
```

```
[?] Specify a domain/IP address to scan: !!!.{}:.x.com

[!] Not a valid domain or IP address! Please input a valid one.
```

2.7. Checking accessibility of remote server

```
268  #~ Checking that the remote server services are actually running, and which ports they are on.
269  nmap $host -Pn -sV -p- > nmapresult
270  sshport=$(cat nmapresult | grep ssh | awk -F/ '{print $1}')
271  ftpport=$(cat nmapresult | grep ftp | awk -F/ '{print $1}')
272  rm nmapresult
273
274  servicechk SSH $sshport
275  servicechk FTP $ftpport
276
277  #~ Checking that we are able to connect to the remote server using the provided details. If not, exit the script.
278  upchk=$(sshp exit 2>&1 | grep denied)
279
280  if [ -n "$upchk" ]
281  then
282
283      echo
284      echo "[!] Remote server IP address, username and/or password is wrong!"
285      echo "[!] Please correct by running: bash rc.sh options or ./rc.sh options!"
286      echo "[!] Exiting..."
287      exit
288
289  fi
290
```

We then check the availability of the remote server services, specifically, SSH and FTP. We use `nmap` for that. `-Pn` allows us to directly scan the server's ports and `-sV` allows us to do a detailed scan to find out what services are on which ports. We also

use the `-p-` option to tell `nmap` to scan all the ports on the server. We do this in case the services are not on the default ports. The results are then saved into a file called `nmapresult`.

The `nmap` results are saved into a file instead of a variable for the next step, which involves manipulating the results to show us the services' ports. This is easier to do when we can manipulate the results as it is output from nmap, complete with new lines. Saving the results to a variable would remove all new lines, making manipulating the text of the results much harder.

The file is then read from using `cat` and the results manipulated by using `grep` and `awk`. The ports for the SSH and FTP ports are then saved into two variables, `$sshport` and `$ftpport`.

The two variables are then passed through the `servicechk` function along with their service names to check if the services are up and available. If it is not, the user is informed that the respective service is not available and the script exits.

```
[!] Remote server SSH service is not running! Exiting...
```

The script then checks the validity of the provided username and password by calling on the `sshp` function and passing the error output to the standard output field using `2>&1`, so the results can be saved in a variable, and filtering the results for the word `"denied"` by using `grep`. This is done as there are more error outputs than having access denied, such as when the user connects to the server for the first time. We are only looking for a permissions denied message.

```
Permission denied, please try again.
```

```
[!] Remote server IP address, username and/or password is wrong!
[!] Please correct by running: bash rc.sh options or ./rc.sh options!
[!] Exiting...
```

The variable is then checked and if it contains the permissions denied message, the user is informed that wrong credentials are provided and the script exits. If the credentials are correct, the script moves onto the next step.

2.8. Connecting to remote server

```
#~ Connecting to remote server and displaying its details.
rsuptime=$(sshp uptime)
rsip=$(sshp curl -s ifconfig.io)
rscountry=$(sshp geoiplookup $rsip | awk -F: '{print $2}' | awk '{$1=""}1')

echo "Uptime: $rsuptime"
echo "IP address: $rsip"
echo "Country: $rscountry"
echo

```

At this point in the script, these are actually the easy parts as we have done most of the legwork in the previous functions. We pass the `uptime`, `curl -s ifconfig.io`, and `geoiplookup` commands to `sshp` and output the results to the user.

We use the `-s` option with `curl` as we do not want `curl` to show anything other than the requested output. `curl` accesses the provided address, in this case, `ifconfig.io`, and returns what is on that address. `ifconfig.io` returns the external IP address of the machine accessing it, which is the remote server here.

As for `geoiplookup`'s output, we pass it through `awk`, firstly with the `-F:` option to define the field separator to be `:`, and tell `awk` to print the second field. We then use `awk` again to empty out the first field with the standard field separator, space, and print out the rest.

```
[*] Input accepted. Connecting to remote server:
Uptime:  12:51:24 up  6:20,  1 user,  load average: 0.00, 0.00, 0.00
IP address: 118.189.213.161
Country:  Singapore
```

2.9. Scans and logging

```
#~ Checking for the existence of the log file. If it does not exist, create and configure one.
if [ -z $(locate /var/log/rc.log) ]
then

    sudo touch /var/log/rc.log
    sudo chmod 666 /var/log/rc.log

fi

#~ Using the remote server to scan the domain/IP address provided by the user and saving the results on both the remote server and local machine.
#~ Also logs the scan into the log file.
scan whois
scan nmap -Pn -sV -F

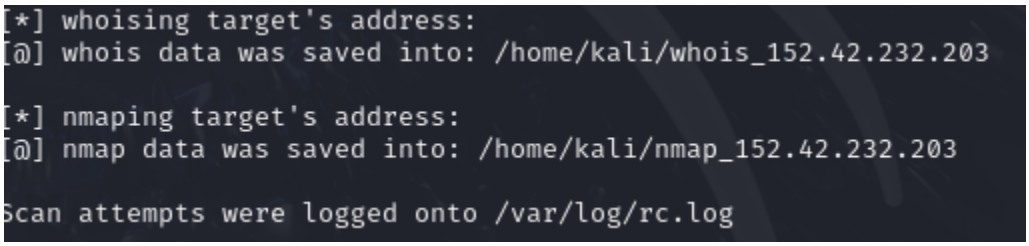
#~ Informing the user the scans were logged.
echo "Scan attempts were logged onto /var/log/rc.log"

```

Lastly, the script checks for the existence of the log file. If it does not exist, it creates one using the `touch` command. It then changes the permissions of the file, using `chmod 666`, so we can write to it.

It then runs the `scan` function and passes on the `whois` and `nmap` command to it. `nmap` is run with `-Pn` and `-sV`, as well as `-F`, so the scan does not take a long time. Using `-F`, however, `nmap` only scans the top 100 popular ports.

The script then informs the user that the scan attempts have been logged and ends.



3. Discussion: FTP

The script above shows that it is pretty easy to automate an attack on a target server. There is one part of the script which I find pretty lacking, which however is a requirement of the script. That part is the requirement to download the scan results from the remote server via an unsecure service. I have chosen FTP here, however, if given the choice, I would have used SCP instead for two reasons.

Firstly, it allows me to stick with the SSH service throughout, and that would mean I only need to check for the availability of the SSH service in this script.

Secondly, using SCP would encrypt my connection between the local machine and remote server, as such it would make it harder for others to find out what I am doing on the machine.

Here, we shall discuss the FTP protocol in detail. FTP, also known as File Transfer Protocol, was introduced in 1971 to facilitate the transfer of files between machines. However, as with most protocols introduced during this era, this protocol's traffic is not encrypted.

FTP works by first having the client send a SYN packet through TCP, which the FTP server then responds with a SYN-ACK packet. The client then acknowledges and the server responds with a 220 code, which indicates the FTP server is ready for the client.

Depending on how the server is configured, the client may need to send the username and password for the server to authenticate, however this exchange takes place in plain text. If the authentication succeeds, the client is granted access to the server and can download files using the `get` or `mget` command. They can also view the list of files and directories on the server using `ls`.

As mentioned, however, all these interactions are unencrypted and people listening in on the exchange using a tool like wireshark can directly just grab all the information exchanged, including the files downloaded and username/passwords used. This seriously impacts the confidentiality side of the CIA triad, as it is really easy to listen in on that exchange, especially if there is any data sent wirelessly.

To solve this issue, there have been two services launched. SFTP(SSH2 File Transfer Protocol), and FTPS(File Transfer Protocol over TLS/SSL). SFTP and FTPS both aim to solve the issue of FTP's traffic being unencrypted, through different means.

In terms of function, they both achieve the aim of providing a secure and encrypted way to transfer files.

SFTP is built off the SSH protocol, which allows it to benefit from SSH's encryption. Due to it being built on SSH, SFTP allows for the client to connect and communicate with the server using one network port. This greatly simplifies how SFTP works, which can lead to less issues with firewalls and file transfers, and reduces the attack surface of SFTP. SFTP is also the newer protocol and may not be supported as well as FTPS in terms of authentication and communications.

Meanwhile, FTPS is still using the FTP protocol, but adds Transport Layer Security(TLS), a cryptographic protocol designed to encrypt communications over a network. FTPS encrypts the traffic using the TLS protocol. The downside of FTPS is that it requires at least two network ports for connection; one to issue commands and separate ports for each directory list or file transfer. This can lead to more issues due to the more complex methods needed. Due to that as well, FTPS also benefits from higher transfer speeds. As mentioned, FTPS is also older than SFTP. Due to that, FTPS has more support from more legacy devices.

4. Conclusion

We can see how easy it is to set up an automated attack using a script. We can also see how easy it is to sniff for information on unsecured protocols. As cybersecurity practitioners, we must be vigilant when sharing information, whether through online or offline means.

The less access people have to our information, the less chance there is of said information leaking out to bad actors. However, that may not always be practical. As such, when we have to share information like passwords and/or usernames, we should do it through encrypted means wherever possible.

For example, using SFTP over FTP when possible is always a good practice as your credentials and files are encrypted through the SSH protocol before being sent out through the network.

We also cannot fall into the thinking that bad actors would never choose us as a target as we are only one person/organization out of many. As seen from this script, we have seen how easy it is to automate an attack. It would only take a little more effort to modify this script so it can be used against multiple targets at once.

As such, when unnecessary, do not open network ports on your machines. For the machines which do require ports to be open, we should set up detection and prevention methods such as limiting the amount tries a user can try to authenticate before they are banned from a service, using tools like fail2ban.

6. References

1. *List of internet top-level domains* (2024) *Wikipedia*. Available at: https://en.wikipedia.org/wiki/List_of_Internet_top-level_domains (Accessed: 30 July 2024).
2. Young, T. (2024) *SFTP vs FTPS - understand the 8 differences and use cases*, *Cerberus FTP Server*. Available at: https://www.cerberusftp.com/blog/ftps-vs-sftp-understanding-the-difference/ (Accessed: 30 July 2024).
3. Writer, C.B.T. *et al.* (no date) *Differences between SFTP and FTPS*, *Spiceworks Inc*. Available at: https://www.spiceworks.com/tech/networking/articles/sftp-vs-ftps/ (Accessed: 30 July 2024).
4. (No date) *What is Transport Layer Security (TLS)? | cloudflare*. Available at: https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/ (Accessed: 30 July 2024).